

# Progressive Retrieval Attention: A Position Paper on Demand-Driven Context Expansion

Jorge Simao

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 2, 2026

## Abstract

Large language models increasingly rely on longer context windows, retrieval-augmented prompts, and agentic search loops to access information beyond their parametric memory. These mechanisms are powerful, but they expose context to the model in a largely flat and eager form: tokens are either placed in the prompt, retrieved once before generation, or acquired through explicit tool calls outside the transformer’s attention computation. This paper argues for a complementary direction: *Progressive Retrieval Attention* (PRA), a family of architectures and runtimes in which lightweight reference handles point to URI-addressed memory and can be expanded on demand into layer-specific attention memory. The central thesis is that long-context inference should move from monolithic token streams toward hierarchical, demand-driven context graphs. We distinguish naming from materialization, selection from transport, and local language ability from reference-path adaptation. These separations position PRA as a bridge between sparse long-context transformers, retrieval-augmented generation, agentic progressive disclosure, and KV-cache runtime systems. We define the abstractions, propose a memory lifecycle and evaluation doctrine, and report only limited feasibility evidence from a two-seed standalone pilot. That pilot suggests a reference path can contribute without modifying a frozen local path, but it does not establish efficiency or general superiority. The position remains a falsifiable research program whose central criterion is improved task quality per unit of context, latency, and memory.

## 1 Introduction

The transformer attention mechanism made it possible for models to condition every token on every earlier token in a sequence [13]. The same mechanism also created a central systems problem: when a model needs access to many documents, code files, logs, prior decisions, or external knowledge fragments, the simplest interface is to concatenate more text into a longer sequence. Modern long-context systems have stretched this interface dramatically, but a larger window does not by itself solve the problem of *selective use*. If only a small fraction of the context is relevant at any generation step, eager inclusion wastes computation and forces the model to perform implicit search inside an expensive dense attention substrate.

Retrieval-augmented generation (RAG) addresses part of this issue by selecting external passages before generation [9, 5, 6]. However, standard RAG often makes a single retrieval decision at the boundary of the model call. The retrieved passages are still inserted as flat prompt text, and the model must consume them whether or not each passage becomes useful later. Agentic systems take the opposite approach: they inspect tool descriptions, issue searches, open files, follow references, and progressively disclose detail only when needed [16, 12]. This behavior is effective but expensive

because the loop is mediated by external actions, repeated model invocations, and hand-designed orchestration.

This paper proposes Progressive Retrieval Attention as a research program for internalizing part of that progressive-disclosure pattern. PRA treats context not only as tokens but as a graph of references. A prompt may contain compact handles such as `<REF_1>`; each handle points through a reference table to a URI such as `mem://manual#installation.cuda`. The runtime can resolve a summary, a fragment, or child anchors. A PRA-enabled layer can then attend to a layer-specific memory cache built from the resolved fragment. The model need not read every referenced document eagerly. Instead, context can be expanded when attention scores, learned gates, uncertainty, or explicit reference-token positions indicate that additional memory is useful.

The paper has four goals. First, it argues that flat long-context scaling, one-shot RAG, and external agent loops occupy different points in a broader design space of memory access. Second, it defines the PRA vocabulary used throughout this research program: reference handles, URI-addressed memory, recursive anchors, layer-specific memory cache, progressive disclosure, and latent context expansion. Third, it separates the architectural, runtime, optimization, and evaluation claims that are often collapsed under the word “retrieval.” Fourth, it lays out falsifiable research questions for implementation papers that follow this position paper.

## 2 Motivation

### 2.1 The Cost of Flat Context

In a decoder-only transformer with dense self-attention, the attention matrix for a sequence of length  $T$  contains  $O(T^2)$  pairwise interactions per layer. Efficient kernels and hardware-aware implementations reduce constants, and sparse-attention variants reduce asymptotic or effective cost [1, 17, 7, 11]. Yet a conceptual mismatch remains: many tasks contain large amounts of potentially relevant context but only a small amount of immediately relevant evidence.

Consider a coding task. A repository may contain thousands of files. A developer rarely reads every file before making a change; they inspect a project map, open candidate modules, follow definitions, and read details near the call sites that matter. The useful information is embedded in a hierarchy: repository, package, file, class, method, block. The flat prompt interface collapses that hierarchy into a token sequence. Once collapsed, the model must recover structure through positional and lexical cues that were never designed as stable memory addresses.

The same pattern appears in scientific QA, technical support, legal analysis, and book-length reasoning. The system often knows that a reference exists before it knows whether the reference must be expanded. A long context window encourages eager expansion because the only available memory interface is token inclusion.

### 2.2 The Shallowness of One-Shot Retrieval

RAG retrieves documents or passages and places them in the model context [9]. This often improves factuality and makes model behavior auditable because generated answers can be traced to retrieved evidence. The limitation is that retrieval is usually performed at one granularity and one time. A retriever may return a passage that is too broad, too narrow, or missing the second-hop information required by the question. The generator then has limited ability to request a more specific subdocument without leaving the forward pass and entering an external loop.

Hierarchical retrieval partially addresses this issue by retrieving coarse summaries before detailed chunks, but the retrieved evidence is still usually materialized as prompt text. The generator

receives the results of a pipeline rather than participating in a continuous memory-selection process. PRA asks whether part of this selection can be made differentiable, layer-local, or runtime-local.

### 2.3 The Expense of Agentic Search

Agentic systems use tools to search, read, execute, and revise. Their strength is not simply tool use; it is progressive disclosure. A capable agent first consumes compact descriptions and only loads detailed instructions, files, or logs when the task demands them. This style is naturally suited to large workspaces and multi-document tasks. Its weakness is cost and latency. Each search or file read may require a model turn, an external call, and a new prompt construction step.

The motivating question for PRA is therefore not whether agents should disappear. Rather, the question is whether the model/runtime boundary can absorb a lower-level version of progressive disclosure: reference-aware memory expansion during inference, with external agents reserved for high-level planning, verification, and action.

## 3 From Flat Context to Context Graphs

### 3.1 Reference Handles

A *reference handle* is a lightweight token or latent marker that names an external memory object without expanding it immediately. In the simplest textual form:

`<REF_1> → mem://doc42#summary`

The handle is paired with a reference table containing at least an identifier, token, URI, summary, and metadata. The prompt can include the handle, the table can expose a human-readable summary, and the runtime can resolve the URI when needed. This separates the existence of a possible context item from its full token cost.

### 3.2 URI-Addressed Memory

PRA assumes that external memory can be addressed by URIs. The scheme may denote local memory, files, repository objects, search results, database rows, or runtime objects:

`mem://manual#installation.cuda.vllm`

The anchor fragment after `#` is not merely a byte offset. It can encode semantic hierarchy: document, section, subsection, paragraph, function, class, or trace event. URI addressing gives the runtime a stable object to cache and gives the model a compact symbol for a potentially large subtree of context.

### 3.3 Recursive Anchors

Many references are best exposed recursively. A root summary may contain child anchors, and child anchors may contain more specific anchors:

`mem://manual#summary → mem://manual#installation → mem://manual#installation.cuda`

Recursive anchors turn context into a graph rather than a bag of chunks. The model can begin with summaries and progressively expand detail. This is particularly important for tasks where the correct evidence is not known from the initial query alone.

### 3.4 Layer-Specific Memory Cache

If an external fragment is encoded into memory for attention, the representation should respect the transformer’s layer structure. A key/value pair computed for one layer is not generally interchangeable with another. PRA therefore uses a layer-specific memory cache:

$$\text{cache}[u, \ell] = (K_{u,\ell}, V_{u,\ell}), \quad (1)$$

where  $u$  is a URI and  $\ell$  is a transformer layer. This distinction matters because lower layers may encode lexical and syntactic information while higher layers may encode task-specific abstractions. A single embedding vector can be useful for retrieval, but it is not a substitute for per-layer attention memory.

## 4 Progressive Retrieval Attention

### 4.1 Conceptual Computation

Let  $X \in \mathbb{R}^{T \times d}$  be the hidden states of a sequence at a given layer. Standard causal self-attention computes

$$\text{SelfAttn}(X) = \text{softmax} \left( \frac{QK^\top + M}{\sqrt{d_h}} \right) V, \quad (2)$$

where  $M$  is a causal mask. A PRA layer augments this with memory selected from a cache. Let  $\mathcal{U}_t$  be a set of URIs selected for the current sequence or position. A simple cross-attention variant computes

$$\text{PRA}(X) = \text{SelfAttn}(X) + \alpha \cdot \text{softmax} \left( \frac{QK_{\mathcal{U},\ell}^\top}{\sqrt{d_h}} \right) V_{\mathcal{U},\ell}, \quad (3)$$

where  $K_{\mathcal{U},\ell}$  and  $V_{\mathcal{U},\ell}$  concatenate layer- $\ell$  memory from selected references, and  $\alpha$  controls the contribution of the memory branch.

The central research problem is selection: how should  $\mathcal{U}_t$  be chosen? Possible triggers include similarity between the current hidden state and summary vectors, attention to explicit reference-token positions, uncertainty in the next-token distribution, learned gates, or runtime policies that trade accuracy against latency.

### 4.2 PRA as Architecture, Runtime, and Research Program

The term PRA is intentionally broader than one neural module. It refers to three coupled objects:

1. **Architecture:** attention layers that can consume external reference memory through cross-attention, KV injection, adapters, or other differentiable mechanisms.
2. **Runtime:** resolvers, caches, URI stores, anchor expansion policies, and scheduling decisions that decide when and how memory is materialized.
3. **Research program:** benchmarks, ablations, scaling studies, and theoretical analyses for demand-driven context expansion.

This distinction helps avoid overclaiming. A standalone research prototype can validate cache construction and attention mechanics without proving that PRA is ready for production-scale pretrained models. Runtime integration can study scheduling and KV memory without resolving all training questions. Scaling theory can ask which tasks benefit from context graphs without committing to a single implementation.

## 5 Conceptual Commitments

PRA is most useful as a research frame when several concerns remain separate. Collapsing them produces systems that may work but cannot be interpreted.

### 5.1 Naming Is Not Materialization

A reference handle states that a memory object exists and can be addressed. It does not imply that the object’s text, embeddings, or K/V tensors have already been loaded. This distinction allows a runtime to expose a large logical context graph while maintaining a small active physical context. It also supports late binding: two deployments may resolve the same logical URI through different stores, permissions, or freshness policies.

The separation creates a useful sequence of representations:

$$\text{handle} \rightarrow \text{metadata} \rightarrow \text{summary} \rightarrow \text{text} \rightarrow \text{layer memory}. \quad (4)$$

Each transition has a cost and can be denied, cached, audited, or deferred. Progressive disclosure is therefore not merely truncation; it is movement through explicit levels of materialization.

### 5.2 Selection Is Not Transport

Selecting a URI and transporting its representation into a model are different operations. Selection may use lexical retrieval, vector similarity, explicit handle positions, learned gates, uncertainty, or an oracle. Transport may use prompt concatenation, cross-attention, adapters, or direct K/V injection. An experiment that changes both at once cannot determine whether a gain comes from finding better evidence or from presenting the same evidence through a better neural interface.

PRA evaluations should cross selection policies with transport mechanisms. For example, oracle selection with cross-attention tests the transport upper bound, while fixed retrieval with alternative transport tests the architectural path. Learned selection should be evaluated only after these simpler controls establish that selected memory can be used.

### 5.3 Language Ability Is Not Reference Adaptation

Adding a memory path can improve a reference-formatted task while damaging ordinary language modeling. Conversely, freezing the local model can guarantee preservation while limiting adaptation. The relevant object is therefore a two-dimensional outcome: reference-task improvement and local-task retention. A useful method should report both, including the exact trainable parameter groups.

This distinction motivates staged optimization. First establish that each architecture learns ordinary language. Then initialize from the matching checkpoint and train only newly activated reference-specific modules. Finally, compare joint fine-tuning under a mixed plain/reference curriculum. Preservation obtained by freezing is a property of the optimization policy, not proof that an architecture is intrinsically immune to forgetting.

## 5.4 Memory Identity Requires Isolation

URI identity is part of the conditioning state. If a batched implementation combines references from unrelated examples without a corresponding mask, it changes the task and can leak evidence across samples. The standalone implementation exposed this issue directly: a batch-union cache made within-batch shuffling ineffective. Correctness required a cache per example. Efficient PRA runtimes must preserve the same logical isolation through cache indices or attention masks even when physical storage and kernels are shared.

## 6 A Reference-Memory Lifecycle

A PRA runtime can be described as a lifecycle rather than a single retrieval call:

1. **Advertise**: expose handles, summaries, types, provenance, and access policy.
2. **Select**: rank candidate handles for a sequence, position, layer, or decoding interval.
3. **Resolve**: map selected URIs to immutable or versioned content.
4. **Encode**: transform content into layer-compatible memory representations.
5. **Admit**: place entries in a bounded active cache under an admission policy.
6. **Attend**: allow authorized sequence positions and layers to consume the memory.
7. **Expand**: follow child anchors when coarse memory is insufficient.
8. **Reuse or evict**: retain useful entries across tokens or requests, or release them under pressure.
9. **Audit**: record which version, URI, layer, and policy influenced the output.

This lifecycle suggests a cost objective. For task quality  $Q$ , prompt tokens  $T_p$ , encoded reference tokens  $T_r$ , latency  $L$ , and active cache bytes  $B$ , a system should be compared on a frontier such as

$$\max Q \quad \text{subject to} \quad c_p T_p + c_r T_r + c_l L + c_b B \leq C. \quad (5)$$

The constants depend on hardware, cache reuse, and service-level objectives. PRA is not successful merely because it shortens the visible prompt. Reference encoding that is repeated for every request may cost more than eager context. The economic case depends on selective admission, amortized reuse, or better quality under a fixed resource budget.

## 7 Architectural Placement

PRA need not occupy every transformer layer. Lower layers often build local lexical and syntactic representations, while later layers may provide more useful queries for semantic memory. A hybrid architecture can therefore place ordinary SelfAttention below PRA-enabled layers. The opposite hypothesis is also plausible: early access to external tokens may be necessary for composition across all depths. Full PRA, upper-layer PRA, mixed blocks, and sparse PRA placement should be treated as competing designs.

The current standalone pilot compares four local layers, four PRA layers, and two local plus two PRA layers. The full PRA pilot is modestly better on reference-conditioned loss, while the hybrid

uses half as many trainable memory output projections and trains faster in phase 2. Two seeds and one scale are not enough to resolve the placement question. A proper comparison needs matched parameter and compute budgets, layer-wise reference attention diagnostics, and tasks stratified by whether required evidence is lexical, syntactic, or semantic.

## 8 Relationship to Existing Work

### 8.1 Long-Context and Sparse Transformers

Longformer, BigBird, Reformer, Routing Transformer, and related models reduce the cost of attending over long sequences by imposing locality, sparsity, hashing, routing, or structured global tokens [1, 17, 7, 11]. PRA is complementary. It does not primarily ask how to attend over a single long sequence more cheaply; it asks whether all candidate context should become part of that sequence in the first place.

### 8.2 Memory-Augmented Transformers

Compressive Transformers store compressed memories of past activations [10]. Memorizing Transformers retrieve nearest-neighbor memories from previous tokens [14]. RETRO retrieves chunks from a large text database and conditions generation on them [2]. PRA shares the ambition of memory-augmented generation but emphasizes explicit reference handles, URI addressing, recursive anchors, and layer-specific cache entries.

### 8.3 Hierarchical Attention and Adaptive Computation

Hierarchical Attention Networks demonstrated the value of multi-level document structure [15]. Universal Transformers and adaptive computation time study dynamic depth or recurrent refinement [3, 4]. PRA can be viewed as adaptive computation over context rather than only over layers or recurrent steps: the system spends more memory bandwidth on references that prove useful.

### 8.4 Retrieval-Augmented Generation

RAG, REALM, and dense passage retrieval moved language models toward non-parametric memory [9, 5, 6]. PRA inherits their motivation but changes the interface between retrieval and generation. Instead of treating retrieval as a pre-generation pipeline stage, PRA treats retrieval products as addressable memory objects that can be expanded and attended to during inference.

### 8.5 Agentic Tool Use

ReAct and Toolformer show that language models can coordinate reasoning and external actions [16, 12]. PRA does not replace tool use. It attempts to make a subset of memory access cheaper and more structured by moving reference resolution and cache access closer to attention.

### 8.6 KV-Cache Runtime Systems

Serving systems such as vLLM’s PagedAttention show that memory layout, paging, and cache management are first-class concerns for modern inference [8]. PRA extends this systems perspective from generated-token KV caches to external reference-memory KV caches. A production PRA runtime would need admission policies, eviction, batching, cache reuse, and memory isolation.

## 9 Illustrative Design Space

Table 1 gives a conceptual comparison. The entries are hypotheses and design claims, not measured results.

Table 1: Conceptual comparison of context-access strategies. This table is illustrative and does not report experimental measurements.

| Strategy          | Context interface          | Strength                        | Failure mode                                  |
|-------------------|----------------------------|---------------------------------|-----------------------------------------------|
| Flat long context | Eager token sequence       | Simple and general              | High cost when relevance is sparse            |
| One-shot RAG      | Retrieved prompt passages  | Good factual grounding          | Retrieval granularity fixed before generation |
| Agentic search    | External tool loop         | Flexible progressive disclosure | High latency and orchestration cost           |
| PRA               | URI-addressed memory cache | Demand-driven latent expansion  | Requires learned/runtime selection policy     |

## 10 Preliminary Feasibility Evidence

A standalone PyTorch pilot now provides limited implementation evidence for parts of this position. Three four-layer, width-256 models were trained on WikiText-2 with the same 2,000-token BPE vocabulary, context length 128, seeds 1 and 7, and 524,288 processed tokens per run. Mean plain-text test losses were 4.7691 for SelfAttention, 4.7457 for full PRA, and 4.7574 for a two-SelfAttention/two-PRA hybrid. The narrow conclusion is that the implemented PRA block retains ordinary language-model functionality when no external memory is present. Same-width parameter counts differ, so this is not a parameter-matched superiority result.

In a second phase, WikiText entries were partitioned into one to five URI-addressed earlier parts and a held-out natural-language tail. For PRA models, the base network was frozen and zero-initialized memory output projections were trained; the SelfAttention formatting control fine-tuned its full network. Valid references reduced mean answer-token loss relative to a disabled memory path by 0.1165 for full PRA and 0.0841 for hybrid PRA. Replacing references by a deterministic cross-example shuffle increased loss by only 0.0145 and 0.0122. The disabled gap shows that the path contributes, while the smaller shuffled gap is the more relevant, but still preliminary, evidence that contribution depends on content.

Plain WikiText loss after phase 2 was unchanged for both PRA variants because the local path remained frozen. The fully fine-tuned SelfAttention control increased plain loss by 0.1859. This comparison demonstrates the value of isolating a new memory path during adaptation; it does not prove that SelfAttention must forget or that PRA always preserves knowledge. The optimization policies differ deliberately.

These measurements do not yet establish the central efficiency thesis. Reference encoding was performed separately for every example on a small GPU, selective triggering was disabled, and cache reuse was not measured. The pilot should therefore be read as evidence that the abstractions can be implemented, trained, ablated, and made content-sensitive, not evidence that PRA outperforms long context or RAG in production.

## 11 Evaluation Doctrine

PRA requires counterfactual evaluation because a single task score cannot reveal how memory was used. At minimum, a promising checkpoint should be evaluated with valid references, a disabled reference path, shuffled references, irrelevant references, empty references, oracle references, and all evidence visible in local context. Targets and prompt structure must remain fixed across these conditions. Shuffling must occur across examples while preserving reference count and approximate length, and batched caches must prevent cross-example leakage.

Four families of outcomes should be reported together:

1. **Task quality:** loss, perplexity, calibrated accuracy, or task-specific utility.
2. **Reference causality:** valid-versus-disabled, valid-versus-shuffled, and valid-versus-irrelevant deltas.
3. **Preservation:** ordinary language and non-reference task quality before and after adaptation.
4. **Resource use:** processed tokens, resolved and encoded reference tokens, synchronized latency, throughput, peak memory, cache bytes, and reuse rate.

Evidence of genuine reference use requires more than a disabled-path gap. A branch can act as a generic domain adapter regardless of supplied content. Valid references should outperform shuffled and irrelevant content, ideally by larger margins on examples proven to be locally insufficient. Reference attention or routing should rank known relevant parts above controls. Claims should be repeated across seeds and reported with matched token, parameter, and compute budgets.

## 12 Research Agenda

The PRA research program should proceed through increasingly demanding stages.

**Standalone controlled models.** Train small decoder-only transformers where all memory objects, reference handles, cache entries, and evaluation labels are controlled. This stage tests whether the basic architecture learns to use reference memory. Immediate work should add at least three seeds, parameter-matched controls, irrelevant and oracle references, distance bins, known necessary evidence, and mixed plain/reference training.

**Pretrained-model integration.** Adapt pretrained Hugging Face models using cross-attention adapters before attempting direct KV injection. This stage should isolate compatibility issues such as rotary position encodings, grouped-query attention, cache layout, and fine-tuning stability.

**Runtime and serving integration.** Study how reference memories are resolved, encoded, cached, paged, shared, and evicted. The central metric is no longer only accuracy, but accuracy per token, per millisecond, and per byte of cache.

**Scaling and theory.** Develop task families where relevant context occupies a controllable fraction of the available corpus. PRA should be evaluated against full-context, sparse-attention, RAG, summary-first RAG, and agentic-search baselines.

**Selection and gating.** Compare explicit reference-token attention, summary similarity, learned routers, uncertainty triggers, and hybrid policies while holding memory transport fixed. Selection should be evaluated through ranking metrics and counterfactual task loss, not only whether an expected URI entered a cache.

**Hierarchical expansion.** Construct benchmarks in which a root summary identifies candidate child anchors but the answer requires one or more deeper fragments. Measure expansion depth, branching factor, missed-evidence rate, and quality per resolved token. Flat single-hop tasks cannot test the progressive part of PRA.

**Training and preservation.** Compare isolated reference-path adaptation, joint fine-tuning, adapters, low-rank updates, and curriculum mixtures. Report both reference improvement and local-task retention. Determine whether zero-initialized memory residuals remain stable at larger scale and whether learned gates avoid permanent saturation.

**Safety, provenance, and access control.** Treat reference resolution as a privileged operation. Study signed or versioned URIs, tenant isolation, policy-aware cache keys, provenance traces, revocation, poisoning detection, and whether the model can infer protected content through shared memory representations.

## 12.1 Falsification Criteria

The research program should be abandoned or substantially revised if repeated, well-controlled studies show any of the following:

- valid references do not outperform shuffled or irrelevant references once formatting and capacity are controlled;
- equivalent prompt concatenation or RAG matches quality at lower end-to-end cost;
- reference encoding and cache management erase any savings from shorter local context, even with reuse;
- learned expansion policies fail to outperform simple oracle-budgeted retrieval or cannot maintain recall;
- local language ability degrades unacceptably under the adaptation needed to use memory;
- URI and cache isolation cannot be enforced under realistic batching and multi-tenant serving.

These criteria make PRA a testable systems-and-model hypothesis rather than a label that can absorb any retrieval mechanism after the fact.

## 13 Risks and Limitations

PRA introduces several risks. First, reference selection can fail silently: if the system does not expand the right anchor, the model may answer from incomplete context. Second, URI-addressed memory creates provenance and security questions. A reference table is a powerful interface; poisoning or misaddressing entries can mislead the model. Third, cache construction may move cost

rather than eliminate it. Encoding references separately is only beneficial when cache reuse, selective expansion, or reduced prompt length outweighs encoding overhead. Fourth, pretrained integration is technically difficult because external K/V must respect layer semantics, position encodings, attention variants, and batching constraints.

Fifth, content-independent branch effects can be mistaken for retrieval. A memory residual may improve a domain simply because it contributes additional activations, even when references are shuffled. Sixth, batching can violate semantic isolation: a physically shared cache must not expose one sequence’s references to another. Seventh, preservation claims can be artifacts of freezing rather than architecture. Eighth, hierarchical expansion can amplify latency variance and create denial-of-service surfaces when references recursively fan out. Finally, stale cache entries create a new consistency problem: a URI, content version, and encoded layer memory must remain aligned.

These limitations motivate careful experimental design. PRA should not be judged by a single accuracy number. It should be evaluated through controlled ablations that measure answer accuracy, expected-reference hit rate, expansion depth, retrieved-token count, full-context token count, latency, cache hit ratio, content-specific counterfactual deltas, plain-task retention, and cache isolation. The current two-seed pilot addresses only a subset of this list.

## 14 Conclusion

Progressive Retrieval Attention proposes a shift in how language models interact with large context. Rather than treating context as a flat token budget to be filled eagerly, PRA treats context as a graph of references that can be named, selected, materialized, attended to, expanded, reused, and audited. The idea draws from long-context transformers, retrieval-augmented generation, memory-augmented models, agentic progressive disclosure, and KV-cache runtime systems. The initial standalone pilot supports feasibility and motivates sharper counterfactual evaluation, but it does not establish the efficiency thesis. The central hypothesis remains precise and testable: for tasks with sparse, hierarchical relevance, demand-driven context expansion should improve the frontier of quality, latency, token processing, and active memory relative to strong long-context, retrieval, and agentic baselines.

## References

- [1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.
- [3] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.
- [4] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.
- [5] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Retrieval augmented language model pre-training. In *International Conference on Machine Learning*, 2020.

- [6] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.
- [7] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.
- [8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023.
- [9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [10] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.
- [11] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. In *Transactions of the Association for Computational Linguistics*, 2021.
- [12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
- [13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [14] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.
- [15] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of NAACL-HLT*, 2016.
- [16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [17] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.